Full Stack Development Interview Assignment

Batch Operations - Listing, Search & Pagination

Objective:

Build a Batch Operations page based on the provided UI reference.
The application should display batch records fetched from a Node.js backend API and allow users to search, filter, and navigate through batch records using pagination.

The objective is to evaluate your basic understanding of:

- React.js
- Node.js
- Express.js
- REST APIs
- Axios / Fetch API
- Frontend & Backend Integration
- Search & Filtering
- Pagination
- Project Structure

Recommended Tech Stack:

Frontend-

React.js

Axios / Fetch API

CSS / Tailwind CSS / Bootstrap

Backend-

Node.js

Express.js

Data Source-

Use a local/mock JavaScript array in the backend to store batch records.

No database integration is required.

Functional Requirements

Batch Listing:

Implement the following

- Create a GET API to fetch all batches.
- Store at least 10 sample records in a local array in the backend.
- Fetch batch records from the backend API.
- Display all batch information in a structured table. Each batch should contain: Batch Code, Course Name, Centre Name, Start Date, Capacity (Enrolled students/ Total Capacity), Status (Running/Upcoming/Completed)

Search Functionality:

Implement a search field that allows users to:

- Search batches by Batch Code.
- Search batches by Course Name.
- Display matching results dynamically.

Filter Functionality:

Implement the following filters:

- Filter batches by Status.
- Filter batches by Centre.
- Provide "All Status" and "All Centres" options.

Search and filters should work together.

Pagination:

Implement working pagination for the batch listing.

Requirements:

- Display 5 records per page.
- Previous and Next buttons should work.
- Page numbers should be clickable.
- Display the current record range and total number of records.
- Pagination should update correctly when search or filters are applied.

Required REST API

Batch APIs

- Get All Batches
- Search Batches
- Filter Batches by Status
- Filter Batches by Centre
- Get Paginated Batch Records

Note: All the above operations can be handled through a single GET API using query parameters

Example: GET /api/batches?page=1&limit=5&search=&status=&centre= The backend should:

- Read records from the local JavaScript array.
- Apply search and filter conditions.
- Apply pagination.
- Return the matching records along with pagination information. Example response:

```
{
  "success": true,
  "data": [],
  "pagination": {
  "page": 1,
    "limit": 5,
    "totalRecords": 10,
    "totalPages": 2
  }
}
```

Frontend Requirements:
Create the following;
UI components based on the provided reference design:

- Batch Listing Table
- Search Input
- Status Filter
- Centre Filter
- Pagination Control

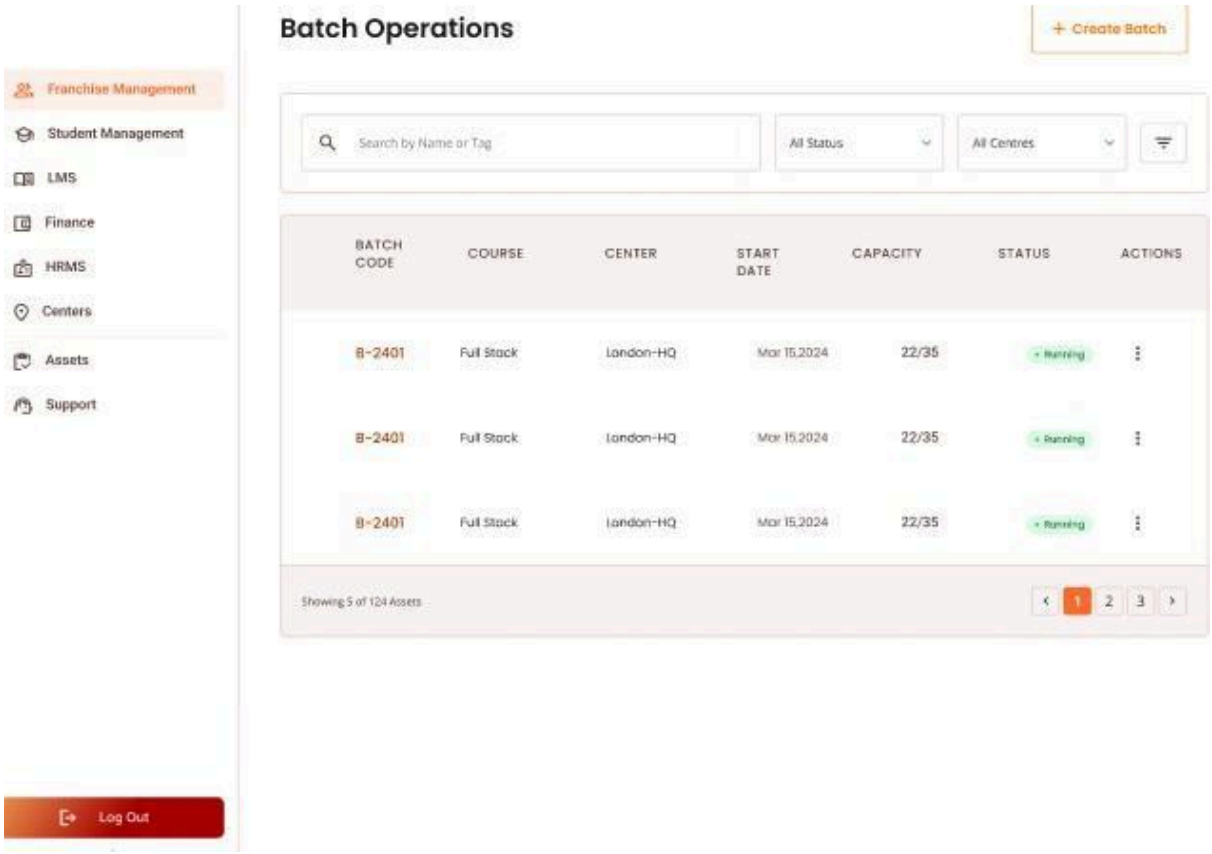
Create Batch Button (UI only; no functionality required).

The table should display all batch information properly.

Also implement:

- Loading state while fetching API data.
- Error handling for API failures.
- Empty state when no matching records are found.

UI-



Suggested Project Structure

project/

```
├── backend/
│   ├── controllers/
│   ├── routes/
│   ├── data/
│   │   └── batches.js
│   ├── app.js
│   └── server.js
│
└── frontend/
    └── src/
        ├── components/
        ├── pages/
        ├── services/
        ├── App.jsx
        └── main.jsx
```

Submission Requirements Submit:

GitHub Repository or Source Code.
README with setup instructions.
Separate frontend and backend code.

Evaluation Criteria:
Timeline- 1 Hour

Notes:

- Follow the provided UI reference as closely as reasonably possible.
- Focus on clean, readable, and maintainable code
- All batch records must be fetched from the backend API, not directly hardcoded in the frontend.
- Database integration is not required.
- Only GET API functionality is required.
- Do not implement POST, PUT, PATCH or DELETE APIs.
- Batch creation functionality is not required.
- You may use any UI library of your choice (Tailwind CSS, Bootstrap, Material UI, etc.).

Best of Luck!